

LLM-Assisted Verification of C Code Generation

Master's Thesis Proposal

Advisor: Zoe Paraskevopoulou

Motivation. CertiRocq's C code-generation pass translates closure-converted, hoisted λ_{ANF} programs into CompCert's Clight language. The proof must relate the evaluation of functional programs to the execution of C code that allocates objects, accesses their fields, and calls functions. Earlier work established correctness of CPS-to-C code generation [1]. The current ANF backend also supports non-tail calls: a caller resumes execution after the callee returns, possibly after garbage collection has moved objects that the caller still needs.

The ANF experience report [2] demonstrated LLM-assisted adaptation of an existing correctness proof. This thesis will explore a further question: can an LLM assist in *developing the logical relation and its invariants*, as well as mechanizing the resulting code-generation proof?

Aim and scope. The aim is to design and mechanize a logical relation between λ_{ANF} and Clight, and use it to prove correctness of the entire current C code-generation pass. The correctness theorem will establish semantic preservation for well-formed input programs, under explicit specifications for runtime operations and assumptions about available memory.

Logical relation and proof. The first task is to define when a λ_{ANF} value and a Clight value represent the same result. This relation depends on the Clight memory: for example, a source constructor with fields must correspond to a pointer to an object whose tag and stored fields represent that constructor. For functions, the relation must express that calls with related arguments produce related results. The student will investigate how to define these clauses together with an expression relation, using step indexing to handle recursion.

The next task is to prove that the generated code satisfies this relation for each source construct. Consider `let y = f(x) in e`. After the generated C call returns, its result must represent the source value of `y`, and the variables needed to evaluate `e` must still represent their source values. If collection occurs during the call, the proof must show that these values were saved on the shadow stack and that the caller reads their updated pointers before continuing. This is one of the new obligations introduced by non-tail calls.

The resulting theorem will apply to every well-formed input for which the executable code generator succeeds, under the stated runtime assumptions.

LLM-assisted methodology. The student will use existing λ_{ANF} logical relations and earlier backend proofs as starting points. An LLM will help propose definitions and auxiliary lemmas, explore proof cases, and develop proofs through Rocq feedback. The student will review the definitions and theorem statements, explain missing arguments, and revise invariants when a proof case reveals a problem. The study will record these revisions and the human guidance needed to complete the proofs. All proofs will be mechanized in Rocq.

Garbage-collector integration: extension or separate thesis. Wang, Stark, and Appel have verified a generational garbage collector using Rocq and the Verified Software Toolchain (VST) [3]. Wang also reports on LLM assistance in completing its proofs [4].

The code-generation proof will initially assume that, when called correctly and with sufficient memory available, the collector preserves the values reachable from the supplied roots, updates pointers to moved objects, and provides space for the next allocation. The compiler proof must still show that generated code supplies all the roots it needs and uses the updated pointers when execution resumes.

The optional extension is to use Wang and collaborators’ proof to establish this assumed property for the actual collector. This requires showing that their theorem about the heap before and after collection implies that each updated root still represents the same λ_{ANF} value. It also requires checking that the generated code’s memory layout and root-saving code satisfy the preconditions of the verified collector.

The resulting code-generation theorem would then use the existing collector correctness theorem in place of an assumption about collection. Establishing this connection requires relating the VST description of the heap to the logical relation used in the compiler proof. This work could be an extension of the proposed thesis or a separate thesis focused on that connection.

Evaluation and expected contribution. The thesis will develop the logical relation and its supporting lemmas, and prove correctness of the full C code-generation pass in Rocq. The evaluation will examine the LLM’s contribution to defining the relation and completing the proof: which definitions required revision, which lemmas could be proved with little guidance, and where the student had to supply a proof argument. The thesis will present the resulting formal development and an account of these decisions.

Indicative six-month plan. Months 1–2: study the backend and set up the logical-relation framework; months 3–4: develop the correctness proof; month 5: investigate integration of the existing garbage-collector proof; month 6: document the proofs and write the thesis. Full GC integration is an extension to the code-generation proof.

References

- [1] O. Savary Bélanger, M. Z. Weaver, and A. W. Appel. *Certified Code Generation from CPS to C*. Manuscript, 2019. Author’s paper.
- [2] Z. Paraskevopoulou. *Machine-Generated, Machine-Checked Proofs for a Verified Compiler (Experience Report)*. PACMPL 10 (ICFP), Article 302, 2026. doi:10.1145/3828700.
- [3] S. Wang, K. Stark, and A. W. Appel. *Verification of a Generational Garbage Collector*. 2026. arXiv:2609.13186.
- [4] S. Wang. *AI-Assisted Completion of CertiGC Proofs: An Experience Report*. 2026. arXiv:2606.21167.